

Ansible Configuration Management

Hands-On Lab Guide: Basic → Moderate → Advanced

+ End-to-End Real-World Use Case

17 Labs | Step-by-Step | Verification Included

Table of Contents

0. Lab Environment Setup

Prerequisites, topology, node preparation

Part 1 — Basic Labs

Lab 1 Install Ansible & Verify Setup

Lab 2 Build Inventory & Test Connectivity

Lab 3 Ad-Hoc Commands

Lab 4 Your First Playbook

Lab 5 Variables & Facts

Part 2 — Moderate Labs

Lab 6 Handlers & Notify

Lab 7 Jinja2 Templates

Lab 8 Loops & Conditionals

Lab 9 Roles & Reusable Structure

Lab 10 Ansible Vault (Secrets Management)

Lab 11 Multi-Environment Configs (group_vars/host_vars)

Part 3 — Advanced Labs

Lab 12 Dynamic Inventory (Cloud)

Lab 13 Custom Modules

Lab 14 Error Handling: Blocks, Rescue, Always

Lab 15 Tags & Selective Execution

Lab 16 Role Testing with Molecule

Lab 17 Ansible Galaxy & Collections

Part 4 — End-to-End Use Case

Deploying a 3-Tier Web Application (LB + App + DB) with CI/CD

0. Lab Environment Setup

Topology

All labs assume one **control node** and up to three **managed nodes**. You can build this using local VMs (VirtualBox/Vagrant), cloud instances (AWS/Azure free tier), or Docker containers (fastest for practice).

Node	Hostname	Role	Suggested OS
Control Node	ansible-control	Runs playbooks	Ubuntu 22.04
Node 1	web1	Managed / Web tier	Ubuntu 22.04
Node 2	web2	Managed / Web tier	Ubuntu 22.04
Node 3	db1	Managed / DB tier	Ubuntu 22.04

Option A: Docker-based lab (fastest, recommended)

```
# Pull an SSH-enabled Ubuntu image and launch 3 "managed node" containers
docker network create ansible-lab

docker run -d --name web1 --network ansible-lab -p 2201:22 rastasheep/ubuntu-sshd:18.04
docker run -d --name web2 --network ansible-lab -p 2202:22 rastasheep/ubuntu-sshd:18.04
docker run -d --name db1 --network ansible-lab -p 2203:22 rastasheep/ubuntu-sshd:18.04

# Default root password for this image is "root" -- change it in production labs
```

Option B: Vagrant-based lab (closer to real VM behavior)

```
# Vagrantfile (3 VMs)
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/jammy64"
  %w[web1 web2 db1].each do |name|
    config.vm.define name do |node|
      node.vm.hostname = name
      node.vm.network "private_network", type: "dhcp"
    end
  end
end
```

```
vagrant up
```

SSH Key Setup (do this once, applies to all labs)

```
# On control node
ssh-keygen -t rsa -b 4096 -f ~/.ssh/ansible_lab -N ""

# Copy public key to each managed node
ssh-copy-id -i ~/.ssh/ansible_lab.pub -p 2201 root@127.0.0.1 # web1
ssh-copy-id -i ~/.ssh/ansible_lab.pub -p 2202 root@127.0.0.1 # web2
ssh-copy-id -i ~/.ssh/ansible_lab.pub -p 2203 root@127.0.0.1 # db1
```

Note: If using Vagrant, SSH keys are auto-injected; use `vagrant ssh-config` to get connection details instead of manual key copying.

Part 1 — Basic Labs

BASIC

Lab 1: Install Ansible & Verify Setup

Duration: 15 min | Nodes needed: Control node only

Objective: Install Ansible on the control node and confirm the version and module path.

1 Update packages and install Ansible

```
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository --yes --update ppa:ansible/ansible
sudo apt install -y ansible
```

2 Verify installation

```
ansible --version
```

Expected output includes the Ansible core version, config file path, and Python interpreter path:

```
ansible [core 2.16.x]
  config file = /etc/ansible/ansible.cfg
  python version = 3.10.x
```

3 Install useful companion tools

```
pip3 install ansible-lint yamllint
ansible-galaxy collection list          # confirm bundled collections
```

Verification: Run `ansible localhost -m ping`. You should see a green `SUCCESS` response using the local connection, confirming Ansible can execute modules.

BASIC

Lab 2: Build Inventory & Test Connectivity

Duration: 20 min | Nodes needed: Control + web1, web2, db1

Objective: Create a static inventory file, group hosts logically, and confirm SSH reachability.

1 Create a project folder

```
mkdir -p ~/ansible-labs/lab2 && cd ~/ansible-labs/lab2
```

2 Create the inventory file

```
# inventory.ini
[web]
web1 ansible_host=127.0.0.1 ansible_port=2201
web2 ansible_host=127.0.0.1 ansible_port=2202

[db]
db1 ansible_host=127.0.0.1 ansible_port=2203

[lab:children]
web
db

[lab:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/ansible_lab
ansible_python_interpreter=/usr/bin/python3
```

3 Test connectivity with the ping module

```
ansible all -i inventory.ini -m ping
```

Expected output for each host:

```
web1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

4 List inventory as Ansible sees it

```
ansible-inventory -i inventory.ini --graph
```

Verification: All three hosts return `SUCCESS` with `"pong"`. If you get `UNREACHABLE`, check SSH port mapping, key path, and `ansible_user`.

BASIC

Lab 3: Ad-Hoc Commands

Duration: 20 min | Nodes needed: Control + web/db group

Objective: Use one-off Ansible commands to perform quick tasks without writing a playbook — useful for troubleshooting and simple bulk operations.

1 Check disk space on all hosts

```
ansible all -i inventory.ini -m shell -a "df -h"
```

2 Install a package (apt module) on the web group only

```
ansible web -i inventory.ini -b -m apt -a "name=curl state=present update_cache=yes"
```

3 Copy a file to all nodes

```
echo "Managed by Ansible" > motd.txt
ansible all -i inventory.ini -b -m copy -a "src=motd.txt dest=/etc/motd"
```

4 Manage a service

```
ansible web -i inventory.ini -b -m service -a "name=cron state=started enabled=yes"
```

5 Gather and filter facts

```
ansible web1 -i inventory.ini -m setup -a "filter=ansible_distribution*"
```

Verification: Each command returns `"changed": true` on first run. Re-running the `apt` command should return `"changed": false` — this demonstrates idempotency.

Module	Purpose
<code>ping</code>	Connectivity test
<code>shell</code> / <code>command</code>	Run raw commands (shell supports pipes/redirects)
<code>apt</code> / <code>yum</code>	Package management
<code>copy</code> / <code>template</code>	File distribution
<code>service</code> / <code>systemd</code>	Service lifecycle management
<code>setup</code>	Gather system facts

BASIC

Lab 4: Your First Playbook

Duration: 25 min | Nodes needed: Control + web group

Objective: Write and run a playbook that installs and starts Nginx on the web group.

1 Create the playbook

```
# site.yml
---
- name: Configure web servers
  hosts: web
  become: true

  tasks:
    - name: Ensure nginx is installed
      apt:
        name: nginx
        state: present
        update_cache: true

    - name: Ensure nginx is running and enabled
      service:
        name: nginx
        state: started
        enabled: true

    - name: Deploy a custom index page
      copy:
        content: "<h1>Deployed by Ansible on {{ inventory_hostname }}</h1>"
        dest: /var/www/html/index.html
```

2 Syntax-check before running

```
ansible-playbook -i inventory.ini site.yml --syntax-check
```

3 Dry-run (check mode)

```
ansible-playbook -i inventory.ini site.yml --check --diff
```

4 Run it for real

```
ansible-playbook -i inventory.ini site.yml
```

5 Confirm via curl

```
curl http://127.0.0.1:2201 # adjust if you mapped an HTTP port instead of SSH-only
```

Verification: Playbook recap shows `ok/changed` counts per host and zero `failed`. Re-running the whole playbook should show `changed=0` across all tasks (idempotency proof).

BASIC

Lab 5: Variables & Facts

Duration: 20 min | Nodes needed: Control + web group

Objective: Parameterize a playbook with variables and use gathered facts inside tasks.

1 Add variables to the playbook

```
---
- name: Configure web servers with variables
  hosts: web
  become: true
  vars:
    app_name: "MyApp"
    doc_root: /var/www/html

  tasks:
    - name: Show gathered OS fact
      debug:
        msg: "{{ inventory_hostname }} is running {{ ansible_distribution }} {{ ansible_distribution_version }}"

    - name: Deploy index page using variables
      copy:
        content: |
          <h1>{{ app_name }}</h1>
          <p>Host: {{ inventory_hostname }}</p>
          <p>OS: {{ ansible_distribution }}</p>
        dest: "{{ doc_root }}/index.html"
```

2 Override a variable from the command line

```
ansible-playbook -i inventory.ini site.yml -e "app_name=OverriddenApp"
```

Verification: The `debug` task prints correct OS facts per host, and the deployed page reflects the overridden `app_name` when `-e` is used.

Variable precedence tip: command-line `-e` vars always win over playbook `vars:`, which win over role defaults — useful to remember once you reach Lab 9 and 11.

Part 2 — Moderate Labs

MODERATE

Lab 6: Handlers & Notify

Duration: 20 min

Objective: Trigger a service restart only when its configuration actually changes, instead of every run.

1 Create a config template and reference it

```
---
- name: Manage nginx with handlers
  hosts: web
  become: true
  tasks:
    - name: Deploy nginx.conf
      template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify: Restart nginx

    - name: Ensure nginx running
      service:
        name: nginx
        state: started

  handlers:
    - name: Restart nginx
      service:
        name: nginx
        state: restarted
```

2 Create a minimal template

```
# templates/nginx.conf.j2
user www-data;
worker_processes {{ ansible_processor_vcpus | default(1) }};
events { worker_connections 1024; }
http {
    include /etc/nginx/mime.types;
    server { listen 80; root /var/www/html; }
}
```

3 Run twice

```
ansible-playbook -i inventory.ini nginx.yml    # first run: changed, handler fires
ansible-playbook -i inventory.ini nginx.yml    # second run: no change, handler skipped
```

Verification: On the second run, the recap shows `changed=0` and the handler does not appear in the output — proving handlers only run on notify.

MODERATE

Lab 7: Jinja2 Templates

Duration: 25 min

Objective: Generate host-specific configuration files dynamically using Jinja2 logic (loops, conditionals, filters).

1 Template with loop and conditional

```
# templates/motd.j2
Welcome to {{ inventory_hostname }}
{% if 'web' in group_names %}
Role: Web Server
{% elif 'db' in group_names %}
Role: Database Server
{% endif %}

Other hosts in this run:
{% for host in groups['lab'] %}
- {{ host }} ({{ hostvars[host].ansible_host | default('n/a') }})
{% endfor %}
```

2 Playbook task

```
- name: Deploy dynamic MOTD
  hosts: lab
  become: true
  tasks:
    - name: Render MOTD from template
      template:
        src: motd.j2
        dest: /etc/motd
```

Verification: `ssh` or `cat /etc/motd` on each node shows a host-specific role and a correctly rendered list of all lab hosts.

MODERATE

Lab 8: Loops & Conditionals

Duration: 20 min

Objective: Install multiple packages with `loop`, and gate tasks with `when`.

```
---
- name: Loops and conditionals
  hosts: lab
  become: true
  vars:
    packages:
      - curl
      - git
      - htop
      - unzip

  tasks:
    - name: Install a list of packages
      apt:
        name: "{{ item }}"
        state: present
        loop: "{{ packages }}"

    - name: Only run on Debian family
      debug:
        msg: "This is a Debian-based host"
      when: ansible_os_family == "Debian"

    - name: Only run on the db group
      debug:
        msg: "Applying DB-only tuning"
      when: "'db' in group_names"
```

Verification: Recap shows 4 package installs looped as separate sub-tasks; conditional tasks are marked `skipped` on hosts that don't match.

MODERATE

Lab 9: Roles & Reusable Structure

Duration: 30 min

Objective: Refactor the `nginx` playbook into a standard, reusable role.

1 Scaffold the role

```
ansible-galaxy init roles/nginx
```

Generated structure:

```
roles/nginx/
  defaults/main.yml    # default variables (lowest precedence)
  files/               # static files to copy
  handlers/main.yml    # handlers
  meta/main.yml         # role metadata/dependencies
  tasks/main.yml       # main task list
  templates/           # Jinja2 templates
  vars/main.yml         # role variables (higher precedence)
```

2 Populate the role

```
# roles/nginx/defaults/main.yml
nginx_doc_root: /var/www/html
nginx_app_name: MyApp

# roles/nginx/tasks/main.yml
- name: Install nginx
  apt:
    name: nginx
    state: present

- name: Deploy config
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: Restart nginx

- name: Deploy index page
  template:
    src: index.html.j2
    dest: "{{ nginx_doc_root }}/index.html"

# roles/nginx/handlers/main.yml
- name: Restart nginx
  service:
    name: nginx
    state: restarted
```

3 Use the role from a playbook

```
---
- name: Apply nginx role
  hosts: web
  become: true
  roles:
    - role: nginx
      nginx_app_name: "ProductionApp"
```

Verification: `ansible-playbook -i inventory.ini site.yml` applies identically to the earlier flat playbook, but the logic is now reusable across projects and testable independently (see Lab 16).

Best practice: One role = one responsibility (e.g., `nginx`, `postgresql`, `firewall`). Compose roles in the playbook rather than building monolithic task files.

MODERATE

Lab 10: Ansible Vault (Secrets Management)

Duration: 20 min

Objective: Encrypt sensitive variables (DB passwords, API keys) so they can live safely in version control.

1 Create an encrypted variables file

```
ansible-vault create group_vars/db/vault.yml
# Prompts for a vault password, opens an editor:
#   db_root_password: "S3cur3P@ss!"
#   db_app_password: "AppP@ss123"
```

2 Reference vault variables in a playbook

```
- name: Set MySQL root password
  mysql_user:
    name: root
    password: "{{ db_root_password }}"
    login_unix_socket: /var/run/mysqld/mysqld.sock
```

3 Run a playbook that needs the vault

```
ansible-playbook -i inventory.ini db.yml --ask-vault-pass
# or, for automation/CI:
echo "myVaultPass" > .vault_pass.txt
chmod 600 .vault_pass.txt
ansible-playbook -i inventory.ini db.yml --vault-password-file .vault_pass.txt
```

4 Edit / view / rekey later

```
ansible-vault edit group_vars/db/vault.yml
ansible-vault view group_vars/db/vault.yml
ansible-vault rekey group_vars/db/vault.yml
```

Verification: Opening `vault.yml` directly in a text editor shows unreadable AES-256 ciphertext; the playbook still resolves the real values correctly at run time.

Security tip: Never commit `.vault_pass.txt` to Git. In CI/CD, inject the vault password via a protected secret variable.

MODERATE

Lab 11: Multi-Environment Configs (group_vars / host_vars)

Duration: 20 min

Objective: Manage dev/staging/prod differences cleanly using inventory-driven variables instead of duplicated playbooks.

1 Directory layout

```
inventories/
  dev/
    hosts.ini
    group_vars/web.yml
  prod/
    hosts.ini
    group_vars/web.yml
```

2 Environment-specific variables

```
# inventories/dev/group_vars/web.yml
nginx_worker_processes: 1
app_env: development

# inventories/prod/group_vars/web.yml
nginx_worker_processes: 4
app_env: production
```

3 Run against a specific environment

```
ansible-playbook -i inventories/dev/hosts.ini site.yml
ansible-playbook -i inventories/prod/hosts.ini site.yml
```

Verification: Same playbook and roles produce different rendered configs per environment purely based on which inventory directory is passed with `-i`.

Part 3 — Advanced Labs

ADVANCED

Lab 12: Dynamic Inventory (Cloud)

Duration: 35 min | Requires: AWS account (or use the provided mock plugin)

Objective: Replace static inventory files with a live query against AWS EC2, so newly provisioned instances are automatically included.

1 Install the required collection

```
ansible-galaxy collection install amazon.aws
```

2 Create the dynamic inventory config

```
# inventory_aws.yml
plugin: amazon.aws.aws_ec2
regions:
  - us-east-1
filters:
  tag:Project: ansible-lab
  instance-state-name: running
keyed_groups:
  - key: tags.Role
    prefix: role
compose:
  ansible_host: public_ip_address
```

3 Test and inspect groups

```
ansible-inventory -i inventory_aws.yml --graph
ansible-inventory -i inventory_aws.yml --list
```

4 Run a playbook against the dynamic group

```
ansible-playbook -i inventory_aws.yml site.yml --limit role_web
```

Verification: Newly launched EC2 instances tagged `Role=web` appear automatically under the `role_web` group without any manual inventory edits.

No AWS account? Practice the same concept with the `docker` inventory plugin (`plugin: community.docker.docker_containers`) against your Lab 0 containers.

ADVANCED

Lab 13: Custom Modules

Duration: 40 min

Objective: Write a minimal custom Python module when no existing module fits a specific need.

1 Create the module folder

```
mkdir -p library
```

2 Write a simple idempotent module

```
# library/app_status.py
#!/usr/bin/python
from ansible.module_utils.basic import AnsibleModule
import os

def main():
    module = AnsibleModule(
        argument_spec=dict(
            path=dict(type='str', required=True),
        ),
        supports_check_mode=True
    )
```

```

    path = module.params['path']
    exists = os.path.exists(path)
    module.exit_json(changed=False, exists=exists, path=path)

if __name__ == '__main__':
    main()

```

3 Use the module in a playbook

```

- name: Check if app marker file exists
  app_status:
    path: /opt/myapp/RELEASE
  register: result

- debug:
  var: result

```

Verification: Running the playbook returns a structured JSON result (`exists: true/false`) exactly like a built-in module, proving Ansible auto-discovered the module from the local `library/` folder.

ADVANCED

Lab 14: Error Handling — Blocks, Rescue, Always

Duration: 25 min

Objective: Build resilient playbooks that recover gracefully from task failures.

```

---
- name: Resilient deployment
  hosts: web
  become: true
  tasks:
    - name: Deployment block
      block:
        - name: Stop nginx before update
          service:
            name: nginx
            state: stopped

        - name: Simulate a risky deployment step
          command: /bin/false    # forces a failure for demo purposes

        - name: This task is skipped due to failure above
          debug:
            msg: "You will not see this"

      rescue:
        - name: Log the failure
          debug:
            msg: "Deployment failed on {{ inventory_hostname }}, rolling back"

        - name: Restore previous release (example)
          command: echo "rollback logic here"

    always:
      - name: Ensure nginx is running regardless of outcome
        service:
          name: nginx
          state: started

```

Verification: Recap shows the block failing, the rescue section executing instead of the play failing outright, and the always section running on every host regardless of success or failure.

ADVANCED

Lab 15: Tags & Selective Execution

Duration: 15 min

Objective: Run only relevant portions of a large playbook instead of the whole thing.

```

---
- name: Tagged tasks example
  hosts: web
  become: true
  tasks:

```

```

- name: Install packages
  apt:
    name: nginx
    state: present
    tags: [install]

- name: Deploy config
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    tags: [config]

- name: Restart service
  service:
    name: nginx
    state: restarted
    tags: [restart, never]

```

```

ansible-playbook -i inventory.ini site.yml --tags config      # only config task
ansible-playbook -i inventory.ini site.yml --skip-tags install # everything except install
ansible-playbook -i inventory.ini site.yml --tags restart    # explicit opt-in, since 'never' excludes by default
ansible-playbook -i inventory.ini site.yml --list-tags       # discover available tags

```

Verification: `--tags config` runs exactly one task; the `never` tagged restart task is skipped unless explicitly called out with `--tags restart`.

ADVANCED

Lab 16: Role Testing with Molecule

Duration: 30 min

Objective: Automatically test the `nginx` role from Lab 9 in an isolated Docker container, verifying it converges and remains idempotent.

1 Install Molecule

```
pip3 install molecule molecule-plugins[docker] docker
```

2 Initialize a test scenario inside the role

```
cd roles/nginx
molecule init scenario -d docker
```

3 Review generated molecule.yml (excerpt)

```

platforms:
- name: instance
  image: geerlingguy/docker-ubuntu2204-ansible:latest
  pre_build_image: true

provisioner:
  name: ansible

verifier:
  name: ansible

```

4 Run the full test cycle

```

molecule create      # spins up test container
molecule converge    # applies the role
molecule idempotence # re-applies and expects zero changes
molecule verify      # runs assertions (e.g., is nginx listening on port 80?)
molecule destroy     # tears down test container
# or simply:
molecule test        # runs the entire lifecycle above in sequence

```

Verification: `molecule test` completes with all stages passing, including a clean idempotence check — this is the same principle CI pipelines use to gate every role change automatically.

ADVANCED

Lab 17: Ansible Galaxy & Collections

Duration: 15 min

Objective: Reuse community-maintained roles/collections instead of reinventing common patterns.

```
# requirements.yml
collections:
  - name: community.general
  - name: community.postgresql
roles:
  - name: geerlingguy.postgresql
```

```
ansible-galaxy install -r requirements.yml
```

```
- name: Use a community role
  hosts: db
  become: true
  roles:
    - geerlingguy.postgresql
  vars:
    postgresql_databases:
      - name: appdb
    postgresql_users:
      - name: appuser
        password: "{{ db_app_password }}"
```

Verification: PostgreSQL is installed and the `appdb` database/user exist, without writing the role's internal logic yourself — confirming community reuse works end-to-end.

Part 4 — End-to-End Use Case

USE CASE

Deploying a 3-Tier Web Application with CI/CD

Duration: 60-90 min | Combines Labs 1-17

Scenario: You are deploying a production-style application consisting of:

- **Load Balancer tier** (HAProxy) — 1 node
- **Application tier** (Node.js app behind Nginx reverse proxy) — 2 nodes, horizontally scaled
- **Database tier** (PostgreSQL) — 1 node, credentials managed via Vault

All configuration is version-controlled, idempotent, environment-aware (dev/staging/prod), and deployed automatically through a CI/CD pipeline on every merge to `main`.

Step 1 — Project Structure

```
ansible-ecommerce/
├── ansible.cfg
├── requirements.yml
├── inventories/
│   ├── dev/hosts.ini
│   └── prod/hosts.ini
├── group_vars/
│   ├── all.yml
│   ├── lb.yml
│   ├── app.yml
│   └── db/
│       ├── vars.yml
│       └── vault.yml          # ansible-vault encrypted
├── roles/
│   ├── common/              # baseline hardening, ntp, users
│   ├── haproxy/             # load balancer role
│   ├── app/                 # Node.js deploy role
│   └── postgresql/          # DB role
├── site.yml                 # master playbook
└── .github/workflows/deploy.yml # CI/CD pipeline
```

Step 2 — Inventory

```
# inventories/prod/hosts.ini
[lb]
lb1 ansible_host=10.0.1.10

[app]
app1 ansible_host=10.0.1.20
app2 ansible_host=10.0.1.21

[db]
db1 ansible_host=10.0.1.30

[all:vars]
ansible_user=deploy
ansible_ssh_private_key_file=~/.ssh/prod_key
```

Step 3 — Master Playbook

```
# site.yml
---
- name: Baseline all servers
  hosts: all
  become: true
  roles:
    - common

- name: Configure database tier
  hosts: db
  become: true
  roles:
    - postgresql

- name: Configure application tier
  hosts: app
  become: true
```

```

roles:
  - app
serial: 1          # rolling deploy: one app server at a time

- name: Configure load balancer
  hosts: lb
  become: true
  roles:
    - haproxy

```

Why serial: 1? It rolls the app tier one node at a time, so app2 stays live and serving traffic through HAProxy while app1 is being updated — a zero-downtime rolling deployment pattern.

Step 4 — Database Role (Vault-backed credentials)

```

# roles/postgresql/tasks/main.yml
- name: Install PostgreSQL
  apt:
    name: [postgresql, python3-psycopg2]
    state: present

- name: Ensure PostgreSQL running
  service:
    name: postgresql
    state: started
    enabled: true

- name: Create application database
  postgresql_db:
    name: "{{ db_name }}"
    become_user: postgres

- name: Create application DB user
  postgresql_user:
    db: "{{ db_name }}"
    name: "{{ db_app_user }}"
    password: "{{ db_app_password }}" # sourced from group_vars/db/vault.yml
    priv: ALL
    become_user: postgres

```

Step 5 — App Role (rolling deploy with health check)

```

# roles/app/tasks/main.yml
- name: Remove app node from load balancer pool
  delegate_to: "{{ groups['lb'][0] }}"
  lineinfile:
    path: /etc/haproxy/haproxy.cfg
    regexp: "server {{ inventory_hostname }} .*"
    line: "server {{ inventory_hostname }} {{ ansible_host }}:3000 check disabled"
  notify: Reload haproxy

- name: Pull latest application release
  git:
    repo: "{{ app_repo }}"
    dest: /opt/app
    version: "{{ app_version }}"

- name: Install dependencies
  npm:
    path: /opt/app
    production: true

- name: Restart app service
  systemd:
    name: myapp
    state: restarted
    daemon_reload: true

- name: Wait for app health endpoint
  uri:
    url: "http://127.0.0.1:3000/health"
    status_code: 200
  register: health
  retries: 5
  delay: 3
  until: health.status == 200

- name: Re-enable app node in load balancer pool
  delegate_to: "{{ groups['lb'][0] }}"
  lineinfile:
    path: /etc/haproxy/haproxy.cfg
    regexp: "server {{ inventory_hostname }} .*"

```

```
line: "server {{ inventory_hostname }} {{ ansible_host }}:3000 check"
notify: Reload haproxy
```

Step 6 — Load Balancer Role

```
# roles/haproxy/tasks/main.yml
- name: Install HAProxy
  apt:
    name: haproxy
    state: present

- name: Deploy HAProxy config
  template:
    src: haproxy.cfg.j2
    dest: /etc/haproxy/haproxy.cfg
  notify: Reload haproxy

- name: Ensure HAProxy is running
  service:
    name: haproxy
    state: started
    enabled: true
```

```
{# templates/haproxy.cfg.j2 #}
frontend http_front
  bind *:80
  default_backend app_pool

backend app_pool
{% for host in groups['app'] %}
  server {{ host }} {{ hostvars[host].ansible_host }}:3000 check
{% endfor %}
```

Step 7 — CI/CD Pipeline (GitHub Actions)

```
# .github/workflows/deploy.yml
name: Deploy Infrastructure
on:
  push:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install ansible ansible-lint
      - run: ansible-lint site.yml

  dry-run:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: ansible-playbook -i inventories/prod/hosts.ini site.yml --check --diff
      env:
        ANSIBLE_VAULT_PASSWORD: ${ secrets.VAULT_PASSWORD }

  deploy:
    needs: dry-run
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - run: echo "$VAULT_PASSWORD" > .vault_pass.txt
      env:
        VAULT_PASSWORD: ${ secrets.VAULT_PASSWORD }
      - run: >
        ansible-playbook -i inventories/prod/hosts.ini site.yml
        --vault-password-file .vault_pass.txt
```

Pipeline gates: `ansible-lint` catches style/logic issues → `--check --diff` dry-run shows exactly what would change → only then does the real `deploy` job run, gated behind a protected `production` GitHub environment requiring manual approval.

Step 8 — End-to-End Verification Checklist

Check	Command	Expected Result
DB reachable from app tier	<code>psql -h db1 -U appuser -d appdb -c '\dt'</code>	Connects, lists tables

App health endpoint	<code>curl http://app1:3000/health</code>	HTTP 200
Load balancer routes traffic	<code>curl http://lb1/</code> repeatedly	Responses alternate between app1/app2
Rolling deploy zero downtime	Run <code>site.yml</code> during a load test	No failed requests during app tier update
Idempotency	Re-run <code>site.yml</code> immediately after success	<code>changed=0</code> across all hosts
Secrets never exposed	<code>git show HEAD:group_vars/db/vault.yml</code>	Shows encrypted ciphertext, not plaintext

Architecture Recap



Where to go next: Add Ansible Tower/AWX for a GUI-driven job scheduler with RBAC and audit logs; integrate `notify` handlers with Slack webhooks for deployment alerts; extend Molecule tests to cover the `app` and `haproxy` roles as well.